

Stock Portfolio Management System

Project Specification

1. Project Overview

This project builds a Stock Portfolio Management System — a desktop application that lets individual investors track the companies they follow, the stocks those companies issue, and every trade they make. It is implemented in Java using Swing for the GUI and JDBC for MySQL database connectivity.

The project is designed to follow exactly the same workflow and application structure as the Music Library project. Students who have completed the Music Library should find the architecture immediately familiar: the same three-tab GUI layout, the same search-then-edit modification pattern, and the same JDBC singleton connection class. The domain and database are different, but the code patterns are identical.

2. Database Design

The database consists of five related entities: Investor, BrokerageAccount, Company, Stock, and TradeTransaction. The Investor entity stores basic information about each user of the system, including InvestorID as the primary key, along with FirstName, LastName, Email, and Phone. Each investor represents an individual who owns and manages stock investments within the system.

The BrokerageAccount entity represents financial accounts used by investors to perform trading activities. Each account has a unique AccountID as its primary key and includes attributes such as AccountType, BrokerName, and Balance. The attribute InvestorID serves as a foreign key linking each account to a specific investor.

The Company entity stores information about companies whose stocks are available for trading. Each company is identified by a unique CompanyID and includes attributes such as CompanyName, Industry, Headquarters, and FoundedYear. The Stock entity represents individual stocks issued by companies. Each stock has a unique StockID and includes attributes such as TickerSymbol, ExchangeName, and CurrentPrice. The attribute CompanyID in the Stock table acts as a foreign key, linking each stock to its corresponding company.

The TradeTransaction entity records all buy and sell activities performed through brokerage accounts. Each transaction is uniquely identified by TransactionID and includes attributes

such as TradeDate, TradeType, Quantity, and PricePerShare. The attributes AccountID and StockID are foreign keys that link each transaction to a specific brokerage account and a specific stock.

3. ER Diagram Requirement

Students must design and submit a complete Entity-Relationship (ER) diagram before beginning implementation. The diagram must include all four entities with their attributes, all primary keys clearly marked, all foreign key relationships with correct crow's foot cardinality notation, and the TRANSACTION table shown as receiving foreign keys from both INVESTOR and STOCK.

The diagram should accurately reflect the relational schema and must be consistent with the implemented tables. Hand-drawn diagrams are acceptable if neat and clearly labeled; using a diagramming tool such as draw.io is strongly recommended.

4. Functional Requirements

4.1 Data Insertion (3 operations)

The GUI must support inserting new records into three tables. For each table, all required fields must be validated before the INSERT statement is executed. The table below lists the required and optional fields for each form.

Sub-tab	Required fields	Optional fields
Investor	Name, Email, Country	Phone
Company	Name, Industry, Country	FoundedYear
Stock	TickerSymbol, Exchange, CurrentPrice, Company (dropdown)	—

The Company dropdown in the Stock form must be populated from the database and refreshed whenever the user switches to that sub-tab. Inserting a Stock requires selecting an existing Company from the dropdown rather than typing a raw ID.

4.2 Data Modification (3 operations)

The GUI must support modifying existing records in the same three tables. The workflow for each sub-tab is identical to the Music Library modification pattern:

- User enters the record ID and clicks Search.
- If the record exists, its current values populate the editable fields and the Update button enables.

- User edits the fields and clicks Update.
- On success, the fields are cleared and disabled until the next search.
- The primary key field (InvestorID / CompanyID / StockID) is always displayed but never editable.

For the Stock sub-tab, the Company field should be presented as a dropdown (same as insertion) so the user can re-assign a stock to a different company without typing a raw ID.

4.3 Data Query (2 operations)

The Query tab must provide two distinct query patterns. Each must display results in a JTable with a row count label. All filters must use PreparedStatement parameters — no string concatenation of user input into SQL.

Query 1 — Transactions by Investor

Element	Detail
Filters	Investor (dropdown), Trade Type (All / BUY / SELL), Sort (Date newest first / oldest first / Quantity high→low)
Tables joined	INVESTOR → TRANSACTION → STOCK → COMPANY
Result columns	Trade Date, Trade Type, Ticker Symbol, Company Name, Industry, Quantity, Price per Share
Notes	Parallel to the 'Songs by Artist' query in the Music Library project

Query 2 — Top N Stocks by Total Trade Volume

Element	Detail
Filters	Industry (dropdown — All or specific), Top N (5 / 15 / 25 / All)
Tables joined	STOCK → TRANSACTION → COMPANY, with GROUP BY and aggregate SUM / COUNT
Result columns	Rank, Ticker Symbol, Company Name, Industry, Total Shares Traded, Number of Transactions
Notes	Parallel to the 'Top N by Annual Sales' query in the Music Library project

5. GUI Requirements

The graphical user interface should be organized into three main sections: Data Insertion, Data Modification, and Data Query. In the Data Insertion section, users should be able to add new records by filling out form-based input fields corresponding to each entity. All

attributes should be represented using labeled textboxes or input controls, and optional fields may be left blank. A submission button should trigger validation and insertion into the database.

In the Data Modification section, users should be able to update existing records by first selecting or entering a record identifier and loading the corresponding data into editable fields. While most attributes can be modified, primary key values must remain fixed to preserve database integrity. An update button should allow users to commit changes, with appropriate validation and error handling.

In the Data Query section, users should be able to interact with the system by selecting filters and parameters using input controls such as dropdown menus and text fields. Queries must be dynamically generated based on user input. Results should be displayed in a table format, clearly showing data combined from multiple related tables.

6. Deliverables

Students must submit the following items:

Please **upload everything** to **GitHub**! Submit your **GitHub link** on **Canvas**.

#	Deliverable	Description
1	ER Diagram	Complete diagram with all entities, PKs, FKs, and cardinalities
2	Relational Schema	Table definitions showing all attributes, keys, and constraints
3	SQL Script	Two-step script: bare CREATE TABLE statements, then ALTER TABLE constraints, plus sample data
4	Java Project	All five source files (DatabaseConnection, GUI, Insertion, Modification, Query panels) as an Eclipse project
5	Screenshots	Clear screenshots of every implemented function (Section 7)

7. Screenshot Requirement

Students must include screenshots demonstrating every implemented functionality. Each screenshot must clearly show the GUI and the result of the operation performed. Specifically, the following must be captured:

- Successful database connection (console output from the TestConnection demo file)
- Inserting a new Investor record — before and after
- Inserting a new Company record — before and after
- Inserting a new Stock record — before and after

- Modifying an Investor record — the Search step, the edit step, and the confirmation message
- Modifying a Company record — same three steps
- Modifying a Stock record — same three steps
- Query 1 (Transactions by Investor) — showing the filter selection and the populated JTable
- Query 2 (Top N Stocks by Trade Volume) — showing the filter selection and the populated JTable

Failure to provide these screenshots will be treated as evidence that the program does not run correctly. **50% of the total project grade will be deducted.**